

Guided Tour Library — Plan & Proposal

A proposal to build a generic, configurable, in-product guided-tour system for DataGenie — one that makes onboarding and release walkthroughs cheap to author (by anyone), stable over time, and reusable across products.

Audience: product & engineering leadership. This document explains *what* we are building, *why*, *which pieces* are involved, and *the choices behind them*. A companion document covers the detailed end-to-end implementation.

Executive summary

Today, every guided tour is hand-built by an engineer: write test IDs into components, write step descriptions and placements into a config file, push code. This repeats for every feature and every release, it locks tour authoring to engineers, and the tours break silently whenever the UI changes. We propose a library that flips this: UI elements expose a small, reusable vocabulary of stable **anchors** that code owns, while the **tours themselves become data** that anyone can author and publish without a deploy. A recorder lets non-technical people build a tour by clicking through a feature; release notes can pre-fill the copy. The result: onboarding and per-release walkthroughs become content work, not engineering work, and every release keeps a durable, replayable tour in a browsable sidebar.

1. Where we are today

We have two needs and one growing cost.

The needs are **onboarding** — a screen-level walkthrough of the major modules for new users — and **release walkthroughs** — a focused tour of what's new each time we ship, kept around so users can replay it later. The cost is **authoring**. An earlier in-repo attempt proved the playback works, but it also exposed the real problem: building a tour means an engineer manually wiring selectors, copy, and steps into a config and pushing code, for every feature and every release. Three consequences follow — only engineers can author, tours rot the moment the UI is refactored, and there is no clean way to keep dozens of release tours alive and browsable over time.

2. What we are proposing

A library, installable in any repo, that delivers:

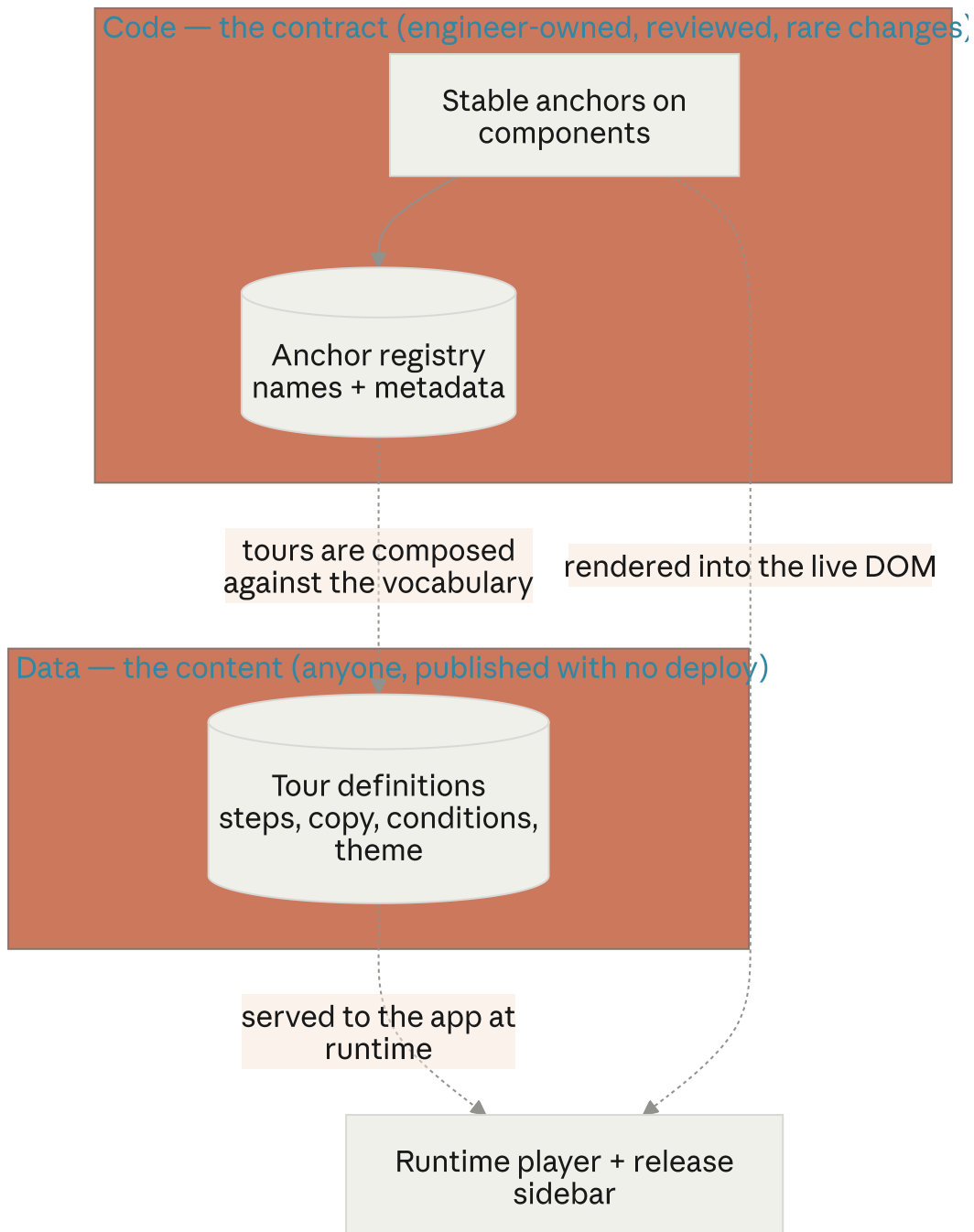
- **Onboarding tours** that span modules with a single configurable flow.
- **Durable release tours** — every release's walkthrough lives in a browsable sidebar with a play button, replayable any time.

- **Authoring by anyone** — a recorder captures a walkthrough as you click through a feature; an LLM can draft the copy from release notes.
 - **Stability over time** — tours don't silently break, because targeting is enforced and validated.
 - **Configurability** — tours are gated by real conditions (route, role, version, feature flag, seen-state), not booleans, and themed to match our design system.
-

3. The idea that makes it work

┃ Code owns the contract. Data owns the content.

A tour is split into two artifacts with two owners and two change rates. The **contract** is a small set of stable anchors on components — engineer-owned, reviewed, slow-moving, and reused across many tours. The **content** — which anchors a tour uses, the copy, order, conditions, theme, and the release list — lives in a backend, authored by anyone, published with no deploy. A tour references anchors by name, so when a feature's elements already carry anchors, building its tour needs **no code change at all**.



The hinge is the **anchor registry** — a build-time catalog of every anchor and its metadata that the recorder, the LLM, the CI checks, and the backend all read. The strategic line: **make the registry the product**. Everything scalable follows from one clean, reusable vocabulary of stable targets.

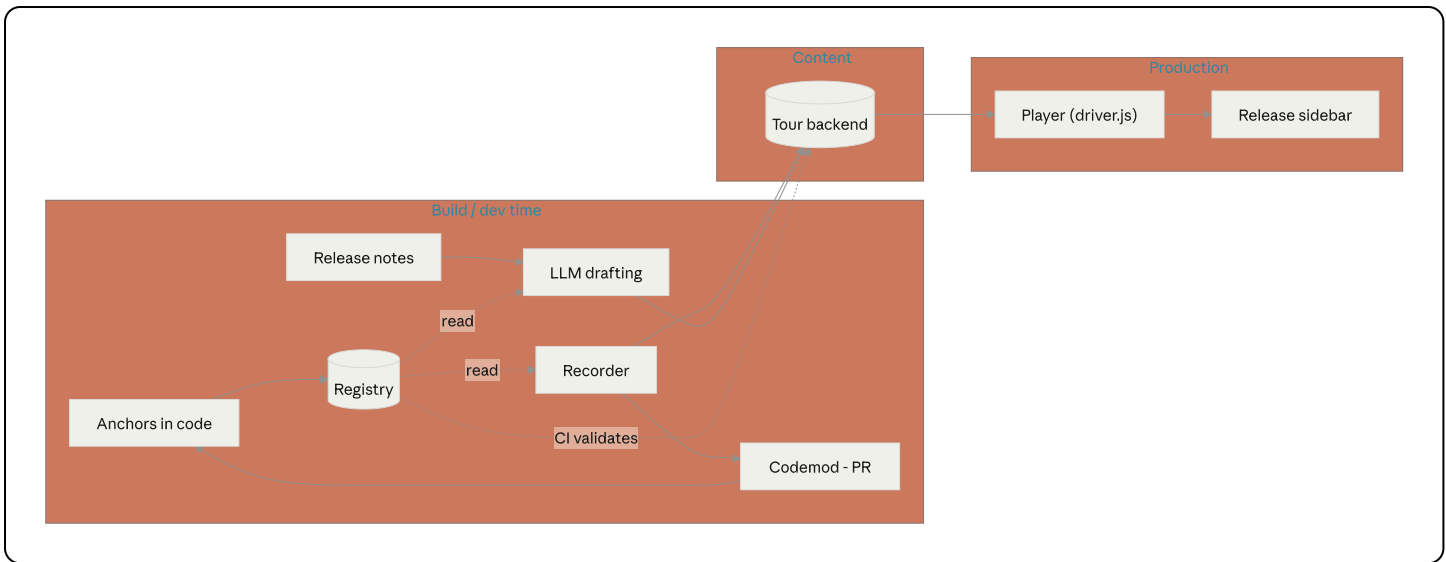
4. What it solves

Pain today	How the library solves it
Engineers hand-wire selectors for every tour	Elements expose reusable anchors; tours reference them by name
Tours break silently on UI changes	Anchors enforced unique at build, type-checked at compile, validated in CI, resolved with fallback + telemetry at runtime

Pain today	How the library solves it
Only engineers can author	Recorder + LLM let non-technical people build tours
Every release needs code pushes	Tour content is data; releases are added/edited without a deploy
No durable, replayable release tours	A release sidebar is a view over the tour backend
Boolean-only control	Declarative conditions: route, role, version range, flag, seen-state, custom
Locked to one app	Runtime ships as installable, framework-light packages

5. The pieces involved

Component	In plain terms
Runtime player	Reads a tour and drives it on screen (highlights, popovers, navigation). Built on driver.js.
React bindings	The provider, the <code>useTour()</code> hook, the anchor helper, and the ready-made release sidebar.
Anchor registry	A build step that catalogs every anchor, enforces uniqueness, and generates type-safe IDs. The vocabulary everything reads.
Recorder	A dev-only overlay: click through a feature, add a description per step, export a tour. Reuses existing anchors; flags gaps.
Codemod	When a gap is found, it adds the missing anchor to the code and opens a reviewable PR.
Tour backend	Stores and serves tour definitions; the sidebar queries it.
LLM drafting	Reads release notes + the registry to pre-fill step copy and suggest which anchors to highlight.



6. Technology choices — and why

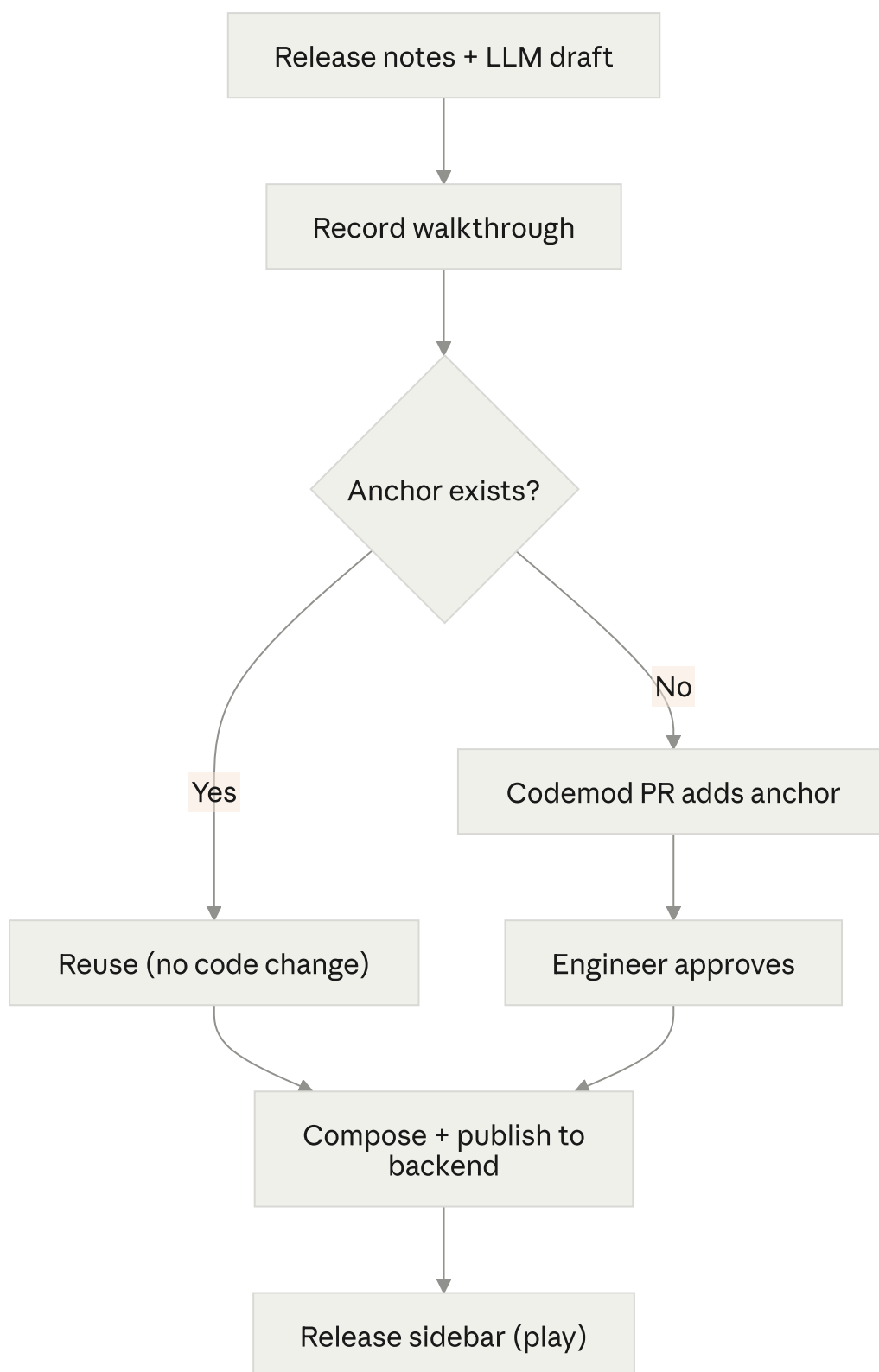
We chose each piece deliberately, after weighing alternatives.

Decision	Choice	Why this	Alternatives considered & why not
Tour player	driver.js	Lightweight, fully themeable, framework-light, MIT-licensed, gives us hook-level control over multi-route flows; already validated in our earlier attempt	Shepherd.js (heavier, extra positioning dependency); Intro.js (commercial licensing cost); React Joyride (React-locked, less control over cross-route tours); build-from-scratch (reinventing positioning/overlay for no gain)
Targeting strategy	Stable (data-tour) anchors	Survive refactors; can be made type-safe and CI-validated	Runtime CSS/text selectors stored in a backend — the no-code SaaS approach (Pendo/Appcues/WalkMe). Rejected because those selectors rot on every refactor, copy change, or i18n, which is fatal for tours that must replay months later
Tour content	Data in a backend	Durable, browsable release sidebar requires editing/adding tours without a frontend deploy	Bundled in-repo config (our earlier attempt) — can't grow a live release list without shipping a build each time
Schema & validation	zod	One source of truth that both validates at runtime and infers TypeScript types	Hand-written types + manual validation (drift between the two); JSON Schema (extra tooling, weaker TS inference)
DOM → source mapping (recorder)	babel jsx-source / React fiber	Standard, reliable way to know which file/line produced a clicked element — the basis for auto-injecting anchors	Heuristic DOM-path guessing (fragile, can't locate source); manual mapping (defeats the purpose)
Anchor injection	jscodeshift / babel codemod	Mature, AST-accurate source edits; can reuse existing anchors and detect prop-forwarding	Regex/string edits (unsafe on real JSX); fully manual (the cost we are removing)
Release-tour versioning	semver ranges	Express "show for versions matching X" cleanly; ties to (package.json)	Hardcoded version equality / booleans (doesn't scale across a release history)

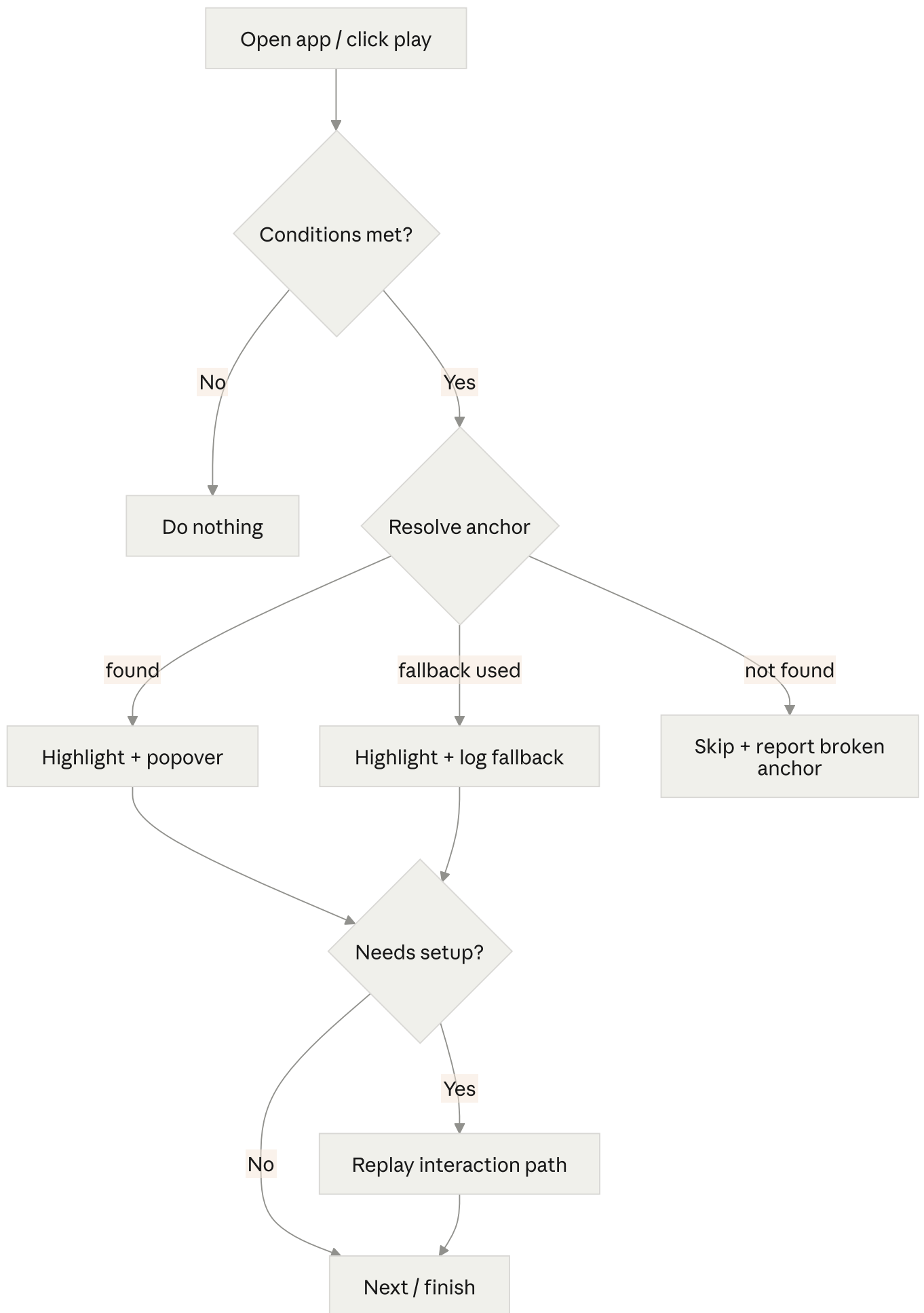
7. How it works, end to end

Authoring a release tour: start from the release notes, let the LLM draft the copy, record the

walkthrough in a dev build. For each target the recorder either reuses an existing anchor (no code change) or opens a small PR to add one. Compose the tour and publish to the backend — no frontend deploy. It appears in the release sidebar.

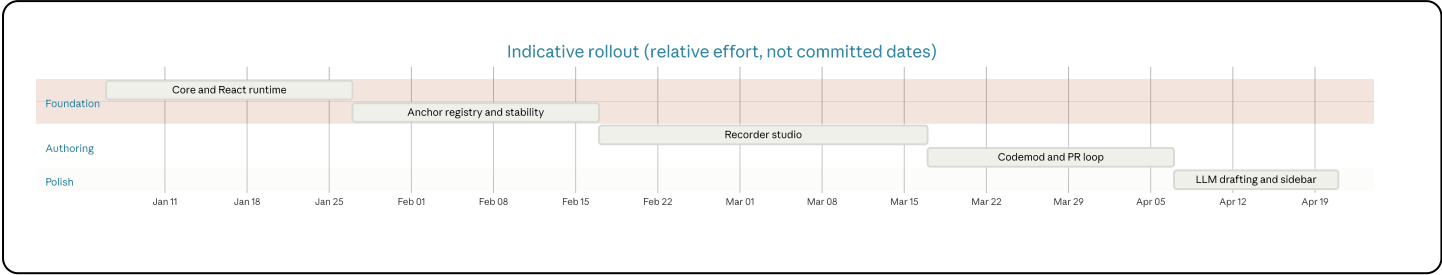


Playing a tour: the runtime checks the tour's conditions, resolves each anchor against the live DOM with a fallback chain, runs any setup path (e.g. open a modal, select a derived KPI), highlights, and records that the user has seen it. If an anchor can't be resolved, the step is skipped and reported — the tour degrades loudly, not silently.



8. Rollout

Runtime-first, so we ship value early and de-risk the hardest parts (codemods on a live codebase) only after the recorder has surfaced their pitfalls.



Phase	Delivers	Value
1	Runtime player, React bindings, conditions, theming (manual config)	Replaces today's approach properly; usable immediately
2	Anchor registry, uniqueness + type safety, resolver fallback, CI rot-check	Tours stop breaking; foundation for automation
3	Recorder (capture, copy, export)	Non-technical authoring; the headline capability
4	Codemod + PR loop	Closes the loop so authors never touch code directly
5	LLM drafting + sidebar lifecycle	Copy drafted from release notes; polished release sidebar

9. Risks & mitigations

Risk	Mitigation
Custom components don't forward props, so an injected anchor never reaches the DOM	Recorder/codemod detect prop-forwarding and mark the anchor unhealthy before it's used
Elements that appear only after interaction (modals, async data)	Recorder captures the interaction path; runtime replays it with wait-for-element
Tours rot as UI evolves	CI validates every published tour against the registry; runtime fallback + telemetry; tour lifecycle (active/deprecated/archived)
Recording can't work in production builds	Recording is a dev-only capability by design (source mapping needs dev instrumentation)

Risk	Mitigation
Low anchor coverage early on	Recorder's reuse-or-PR loop shrinks gaps with every tour authored
Multi-language copy	Tour copy stored keyed by locale in the backend

10. What success looks like

- A growing share of release tours that need **zero code changes** to author.
- **Minutes, not a sprint**, to author a release tour — and by a PM/marketer, not only an engineer.
- **Broken tours caught in CI**, before users ever see them.
- A **browsable history** of release walkthroughs users can replay on demand.
- The runtime reusable across DataGenie products and beyond.